

FamilyHub RLS policies

Resumen

- ? Archivo de migracion creado:
- ? backend/sql/migration_002_rls_auth.sql
- ? Archivo de test creado:
- ? backend/sql/rls_test.sql
- ? Tablas afectadas:
- ? public.grupos_familiares
- ? public.miembros_grupo
- ? public.perfiles

Policies creadas

- ? grupos_familiares
- ? select por membresia compartida
- ? insert solo con creador_id = auth.uid()
- ? update solo creador
- ? delete solo creador
- ? miembros_grupo
- ? select por household compartido
- ? insert por coordinador real del enum o bootstrap seguro del creador
- ? update solo coordinador
- ? delete por coordinador o por el propio miembro
- ? public.perfiles
- ? select solo si comparte hogar con el target
- ? update solo self
- ? delete bloqueado por ausencia de policy

Decisiones tecnicas

- ? La migracion elimina policies previas en las tres tablas objetivo antes de crear las nuevas.
- ? Se agregaron funciones helper security definer para evitar recursion infinita entre policies que necesitan consultar household_members.
- ? Se usa directamente 'ADMINISTRADOR':public.rol_familiar como rol coordinador real del schema actual.
- ? No active force row level security por no tener el schema real versionado ni contexto suficiente de mantenimiento.

- ? El creator bootstrap en miembros_grupo se habilito para que el creador pueda insertar su propio rol coordinador real despues de crear el hogar.
- ? El caso "self join por token" no se habilito en RLS puro porque el row insertado no transporta el token de invitacion y no hay forma segura de demostrarlo solo con la fila.

Limites detectados

- ? El schema real no esta versionado en el repo.
- ? No pude auditar policias remotas ni grants reales desde esta sesion por bloqueo de red.
- ? Con los CSV confirmamos el schema real:
- ? grupos_familiares
- ? miembros_grupo
- ? perfiles
- ? La migracion asume que public.perfiles.id = auth.users.id.

Como correr rls_test.sql

1. Aplicar primero backend/sql/migration_002_rls_auth.sql.
2. Ejecutar backend/sql/rls_test.sql en Supabase SQL Editor o psql con un rol privilegiado.
3. El script corre dentro de transaccion y termina con ROLLBACK.

Que valida el test

- ? crea user_a
- ? crea user_b
- ? crea un hogar por usuario
- ? crea el coordinador inicial para cada hogar
- ? valida que user_a si puede ver su hogar
- ? valida que user_a no puede ver el hogar de user_b
- ? valida que user_a si puede ver su perfil
- ? valida que user_a no puede actualizar el perfil de user_b

Riesgos menores

- ? Si authenticated no tiene grants sobre las tablas, el test puede fallar antes de ejercer RLS.
- ? Si el enum public.rol_familiar cambia y deja de usar ADMINISTRADOR, hay que ajustar el literal casteado en la migracion y en el test.
- ? La regla de union por invitacion debe seguir reforzada por backend mientras el insert no lleve una prueba verificable del token.